

# Installation Instructions for InvoiceLab

---

Suthep “Jogie” Madarasmi, Ph.D.

# LAB 1. Get InvoiceLab installed and running

---

## Goal

- Run DB + API + UI
- Explain request flow

## Deliverable

- Screenshot of working app

## Teaching Focus on Mental Model First:

UI → API → DB → API → UI

## Parts in this Slide

---

1. Introduction to Docker and Containerization
2. Optional: Installation on a new Windows virtual machine via hyper-V
3. System software installation on a Windows PC (can be VM)
4. Starting up the 3-Tier InvoiceLab App
5. Test run of the InvoiceLab App

## **Part 1.**

# **Introduction to Docker and Containerization**

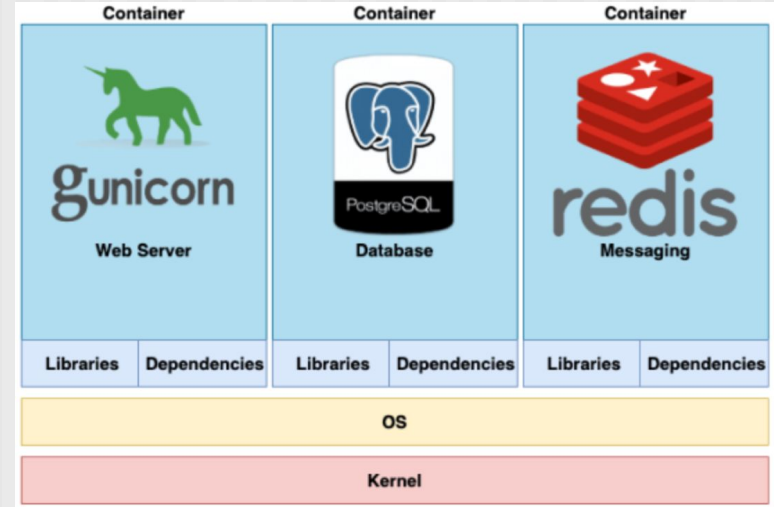
---

# What is Docker?

Docker is a container platform that allows us to package:

- An application
- Its runtime
- Its libraries
- Its system dependencies

Into a single executable unit called a **container**.



A container runs the same way **on any machine** with Docker installed.

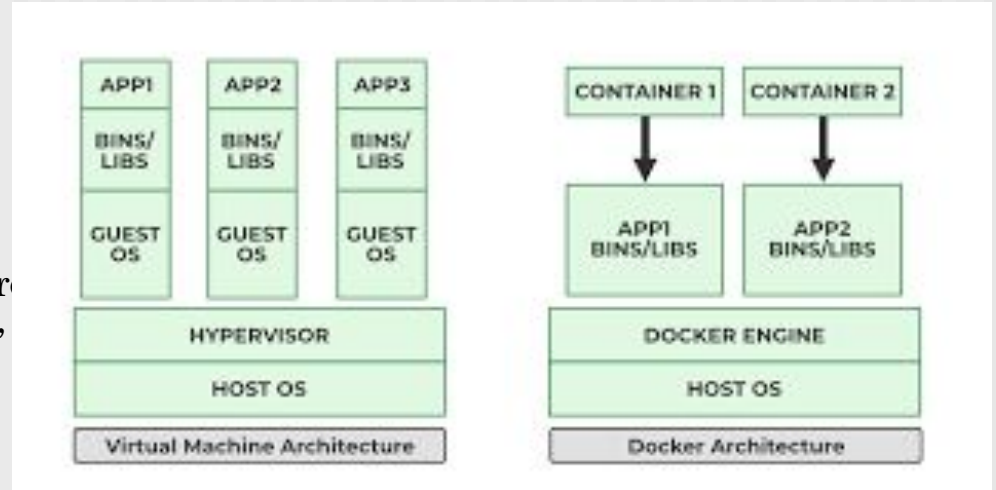
# Container vs. Virtual Machines (VMs)

## Virtual Machine (VM):

- Has its own full OS
- Heavyweight, Slower to start
- Runs via hypervisor. A Management software layer that lets you run multiple “computers” or VMs inside one real computer or PM.

## Docker Container:

- Shares host OS kernel
- Lightweight, Starts in seconds
- Runs as an isolated process



# How Can Linux Containers Run on Windows?

---

Most containers (including PostgreSQL) are based on **Linux images** .

But Windows cannot run Linux binaries directly.

So what happens?

Docker Desktop uses:

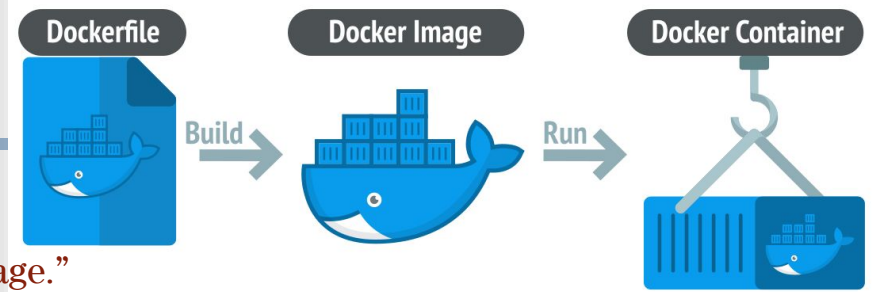
- **WSL2 (Windows Subsystem for Linux) or** Another lightweight hidden Linux VM

Inside that Linux layer:

- The container runs normally.
- Your Windows machine interacts with it.

You don't see the Linux OS directly — Docker manages it.

Dockerfile → builds → Docker Image  
→ runs as → Docker Container



1. A **Dockerfile** is a text file with instructions to build an image. “A recipe for building a Docker image.”
  - Starting base image (e.g., **FROM node:20 - minimal Linux + Node**)
  - Add your app files, Dependencies to install, Commands to run to start it
2. A **Docker image** is a built, read-only template created from a Dockerfile. “A snapshot or blueprint”. Images are used to create running containers.
  - Application code, Runtime, System libraries, Configuration
3. A **Docker container** is a running instance of an image. “A running application created from an image”.
  - An isolated process
  - Own filesystem layer
  - Uses host OS kernel
  - With network and resource limits
  - Can start, stop, delete a container

#### Example: PostgreSQL Docker

**Dockerfile.** Defines how Postgres is packaged

**Image.** `postgres:15`

**Container.** A running DB server on `localhost:15432`



# What Actually Runs on Your Machine

When you install Docker Desktop:

1. Docker installs a lightweight Linux runtime (via WSL2).
2. Containers run inside that Linux environment.

From the user perspective:

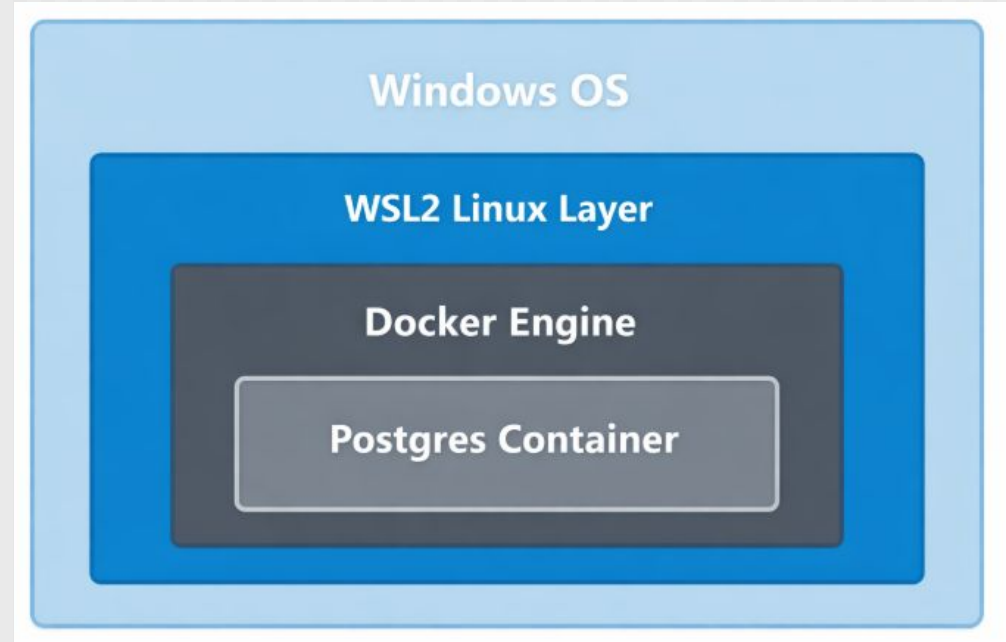
- You run `docker compose up`
- PostgreSQL starts
- It appears at `localhost:15432`



But internally: Windows → WSL2 Linux → Docker → Container → PostgreSQL

# Docker container architecture

- Docker can run both Linux and Windows containers.
- Most dev uses Linux containers, even on Windows or macOS.
- On Windows, Docker uses Windows Subsystem for Linux version 2 (WSL2) to run Linux containers. WSL2 gives you:
  - Linux inside Windows
  - Very fast startup
  - Shared filesystem access
- Docker allows us to run a Linux-based database server inside a hidden Linux layer (WSL2), without installing the database directly on Windows.



# Why We Use Docker in This Course

---

Without Docker:

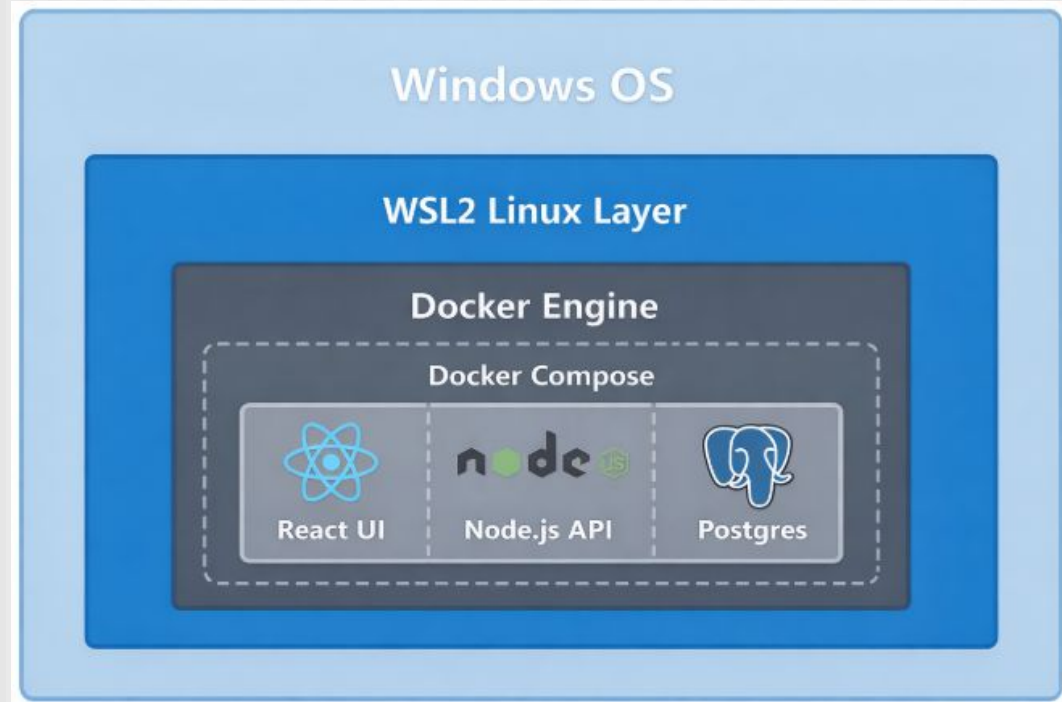
- Students install PostgreSQL manually
- Version conflicts happen
- Setup errors multiply

With Docker:

- Same Postgres version for everyone
- Same config
- Reproducible environment
- Easy reset

It isolates infrastructure from your main OS.

# Docker container / Docker Compose architecture



# Docker Compose: containers with shared data (volumes) & network

Upon PC Restart:

> `docker compose up`

Two things happen:

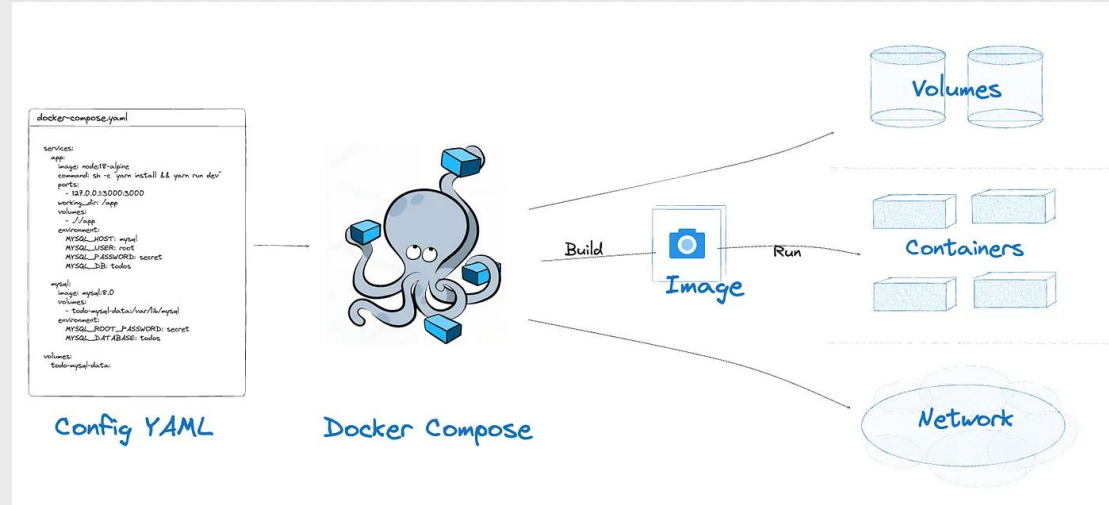
1. **Container is recreated or restarted**
2. **Volume is reattached**

The actual database files live in the **Docker volume**, not inside the container's temporary filesystem.

**When Would Data NOT Persist?**

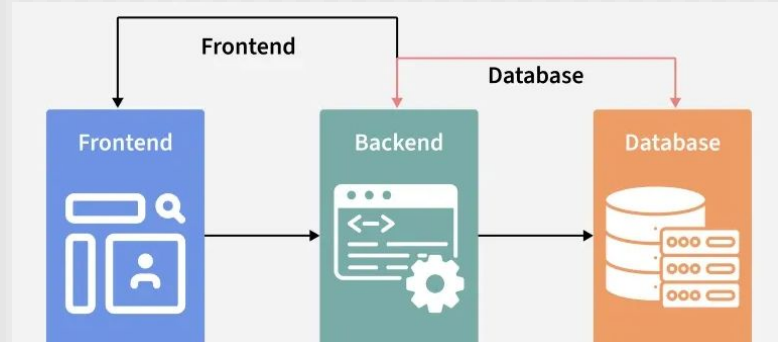
When you manually remove it, such as:

> `docker compose down -v`

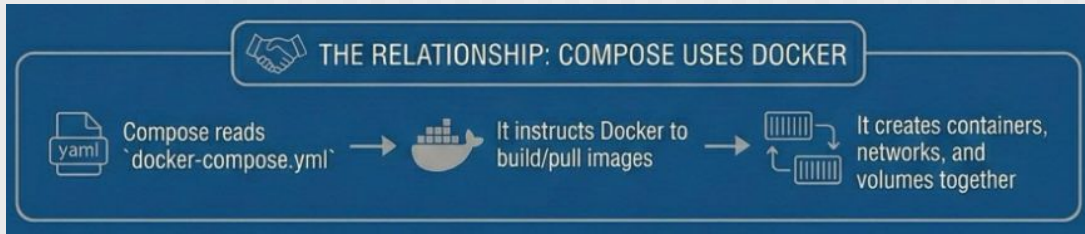


# We can run full-stack (3-tiers) in Docker

1. Alternative mode starts DB + server + client containers
2. **Start:** `npm run docker:start`
3. **UI:** `http://localhost:3000`
4. **API:** `http://localhost:4000`
5. **Stop:** `npm run docker:stop`



Docker Compose environment with 3 docker containers. Compose does the orchestration (coordination).



## Even with one container, Docker Compose is good practice because it keeps configuration clean, reproducible, and version-controlled.

If you only need one container, you *can*:

```
> docker run -d -p 15432:5432 -v pgdata:/var/lib/postgresql/data postgres
```

But now imagine teaching students to remember:

- ports, environment variables
- volume mappings, restart policy, container name

### What Compose Gives You (Even for One Container)

With a simple `compose.yaml` configuration file.

```
> docker compose up -d
```

#### Benefits:

- Version-controlled configuration
- Self-documenting setup, Easy to modify later
- Easier for beginners, Cleaner reproducibility

#### Put into configuration file `compose.yaml`:

```
services:
  db:
    image: postgres
    ports:
      - "15432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: root
```

**Real-World Practice:** Compose is used even for single services and Infrastructure-as-code is preferred.

## Part 2.

**Optional:** Install a new Windows Virtual Machine via Hyper-V

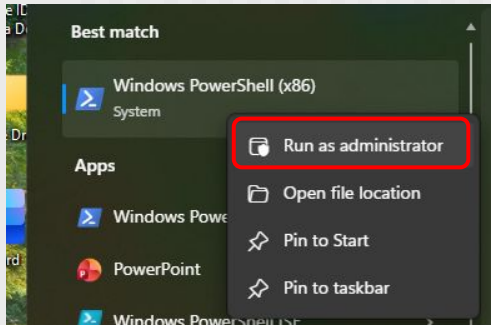
---



# Installation on a Hyper-V VM is optional

---

- To make this video we need a blank PC with no pre-installs
- Instead of uninstalling everything I have on my PC, I decided to create a new VM
- Students may want to run their assignment on a VM on their Laptop as well
- If students prefer to install InvoiceLab directly onto their Laptop they can skip this Part 2.
- If students wish to create a VM to run InvoiceLab on that virtual PC, continue with this section.



1. Run Windows PowerShell as Administrator
2. Enable Hyper V optional feature and then reboot.

```
Select Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

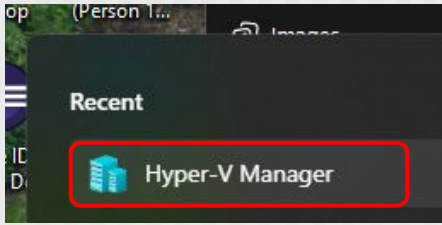
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

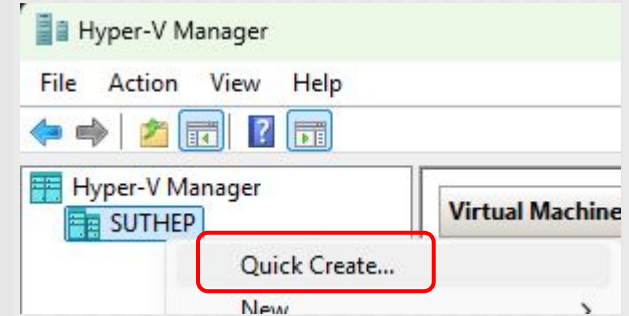
> Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All

(Then Reboot)

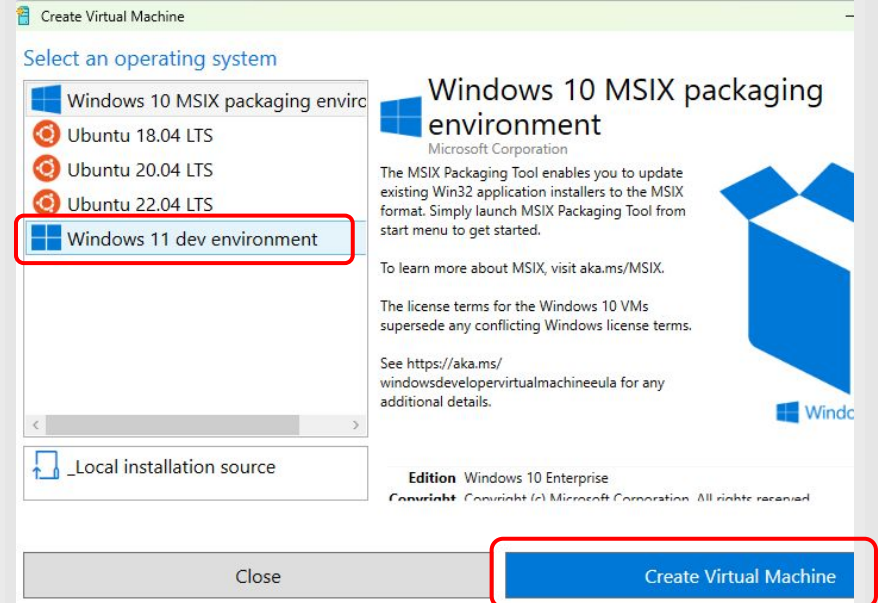
### 3. Run Hyper-V Manager



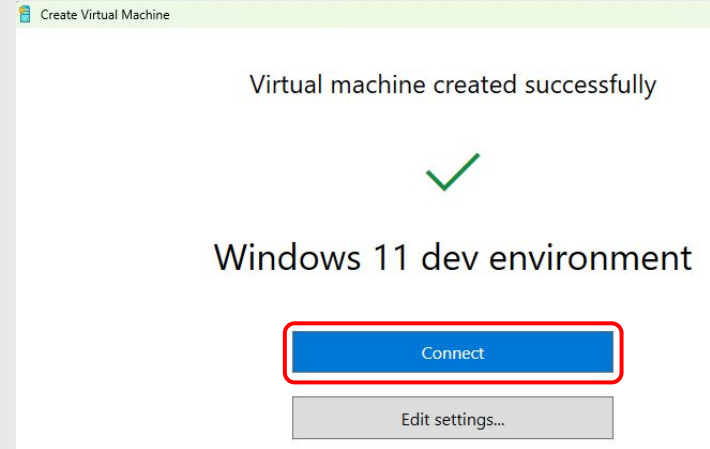
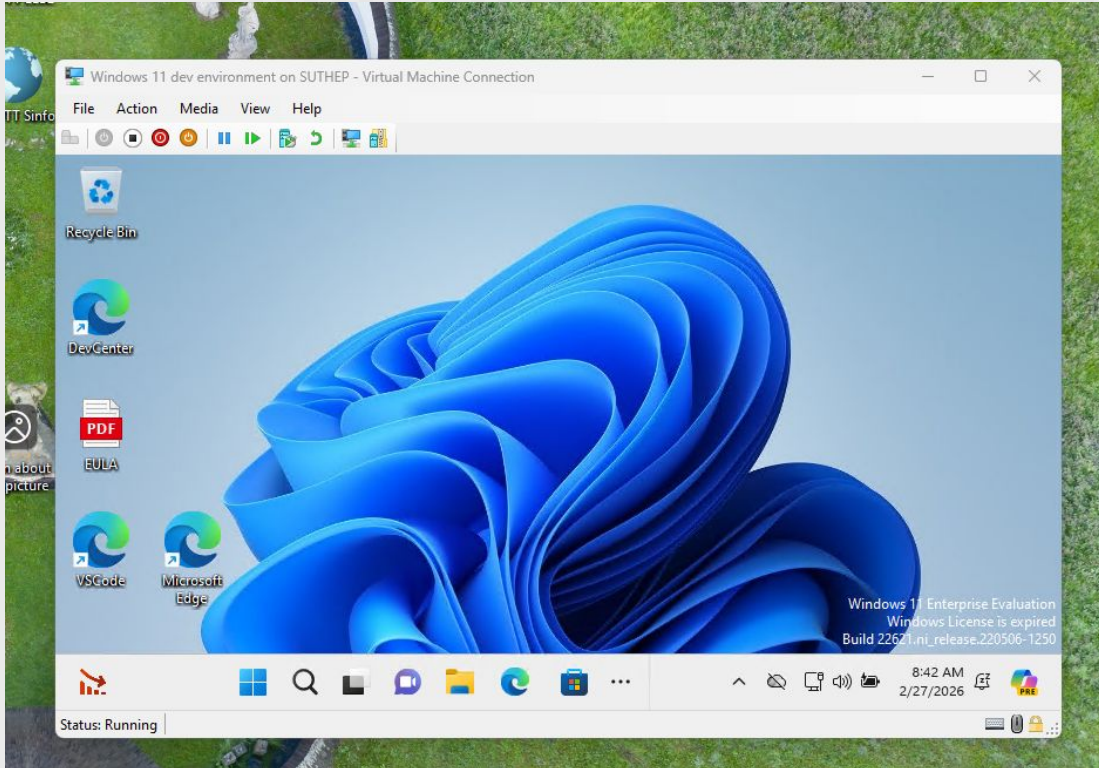
### 4. Under your PC Name, select Quick Create ...



### 5. Select Windows 11 dev environment and [Create Virtual Machine]



6. After Virtual machine created successfully message press [Connect] to start your virtual machine.



# To run Dockers: Must enable virtualization in the Hyper-V VM

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

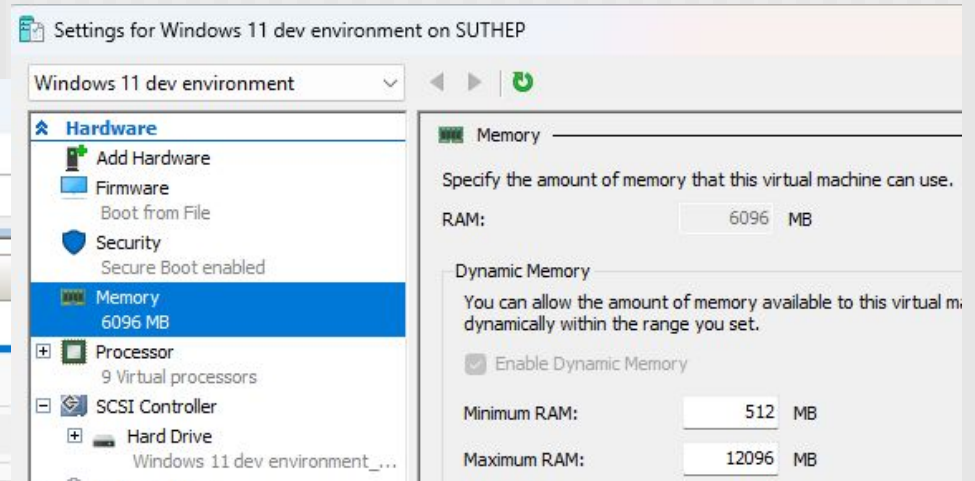
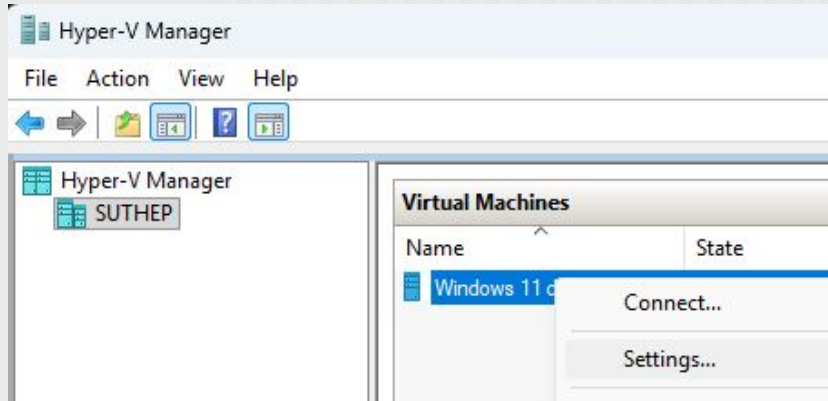
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Set-VMProcessor -VMName "Windows 11 dev environment" -ExposeVirtualizationExtensions $true
PS C:\WINDOWS\system32>
```

## 7. In main PC's Powershell:

> Set-VMProcessor -VMName "Windows 11 dev environment" -ExposeVirtualizationExtensions \$true

## 8. In Settings of VM Increase the Memory



## Part 3.

# System software installation on a Windows PC

---

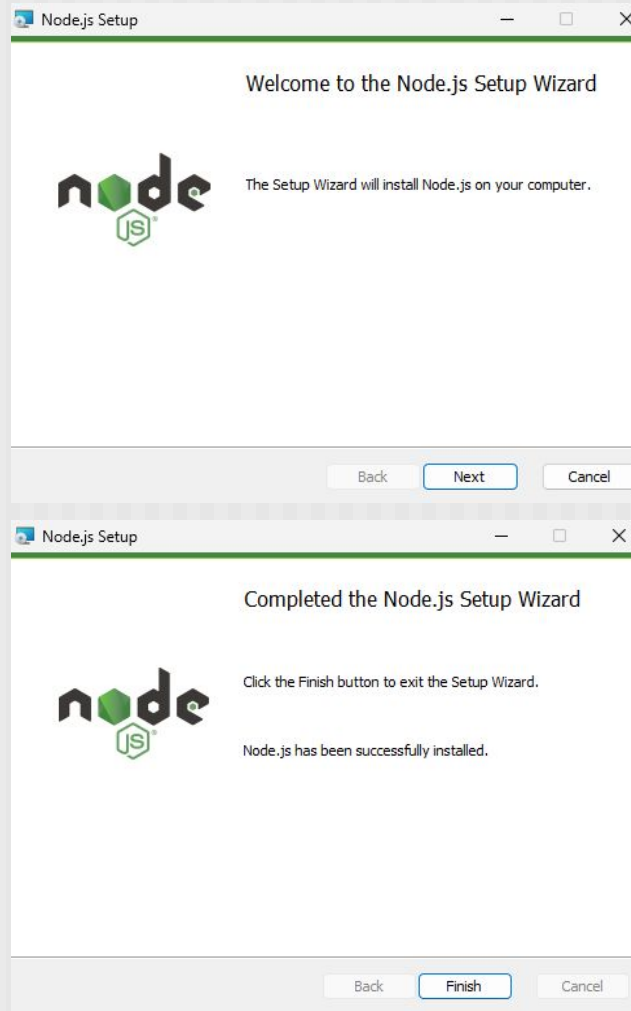
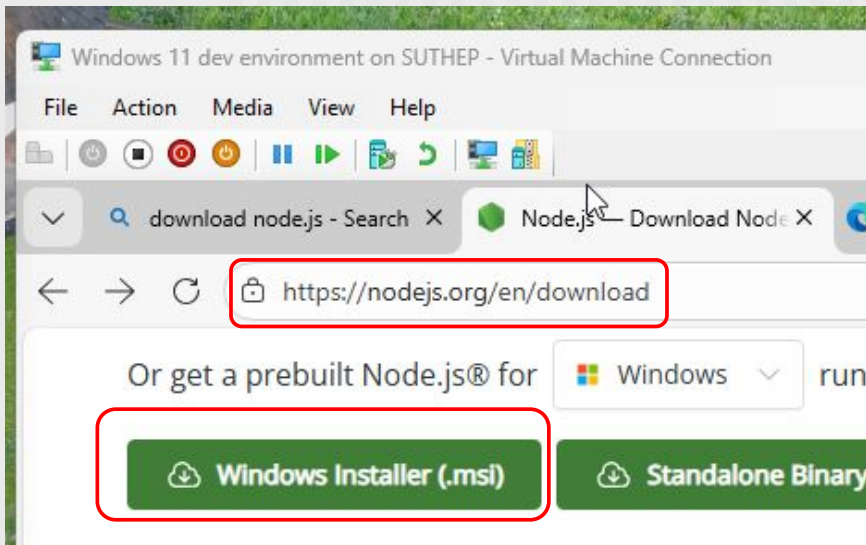
This PC can be a VM or a PM

## New machine preparation (install these)

---

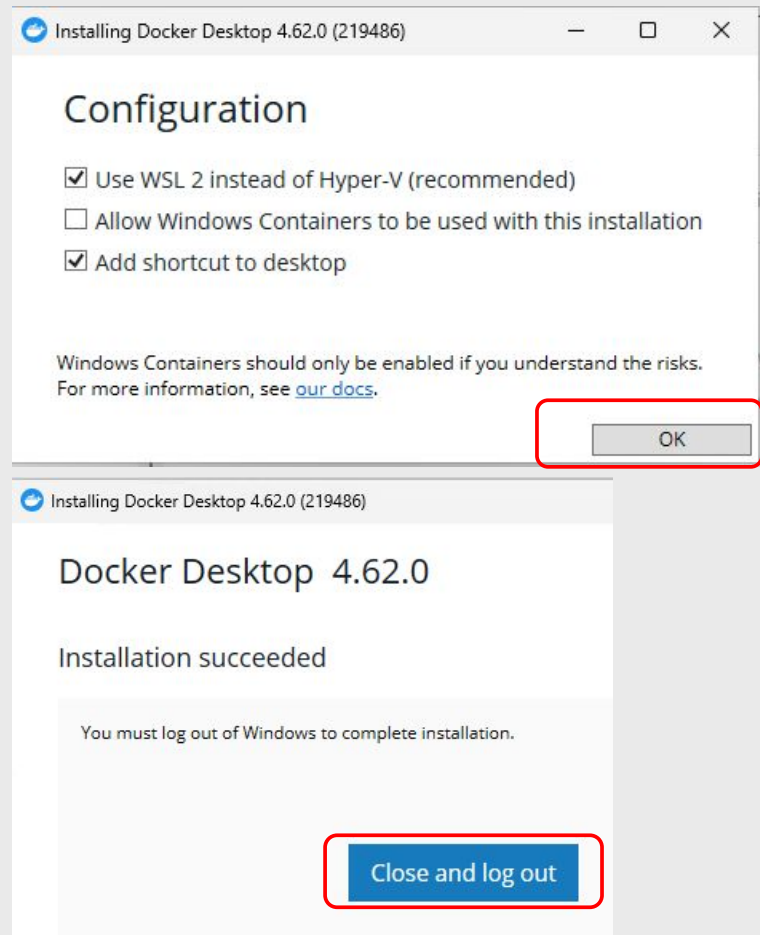
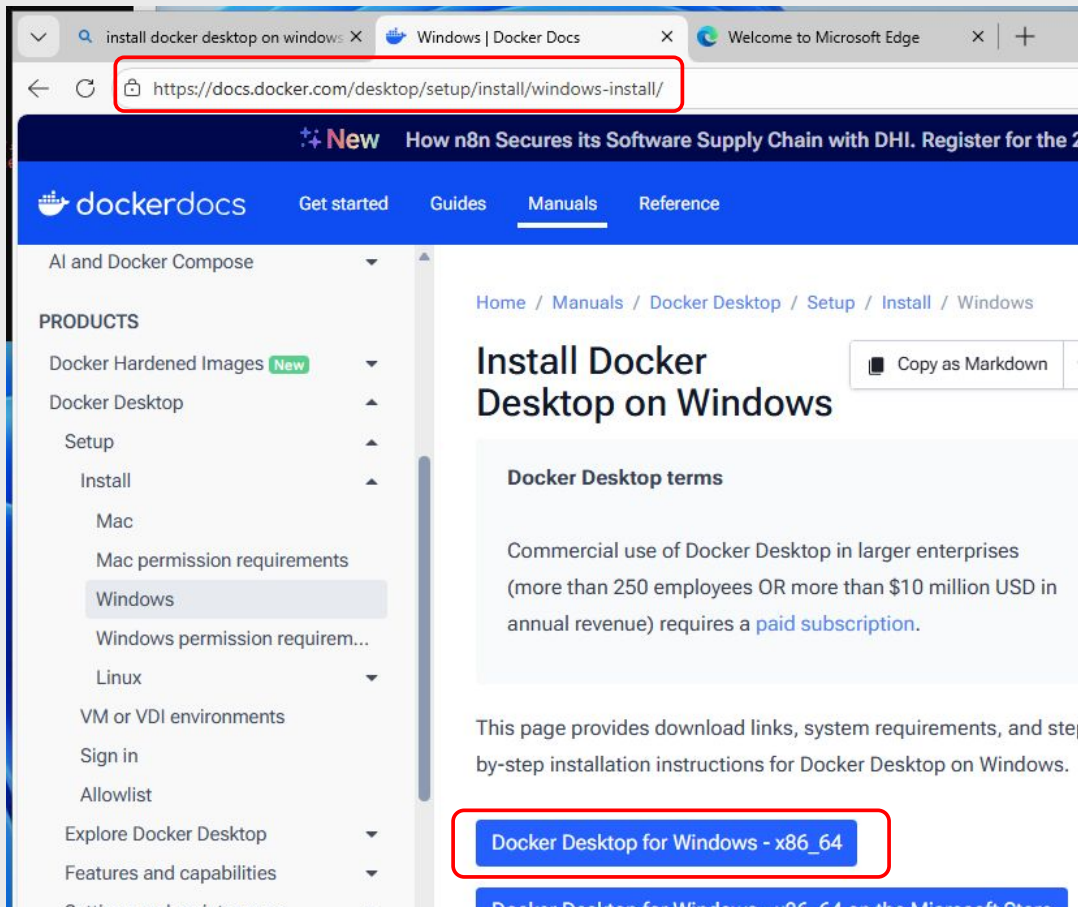
1. Node.js 20.19+ (Windows LTS .msi; ensure “Add to PATH”)
2. Docker Desktop (to use Linux containers)
3. Install Visual Studio Code (VS Code)

Recommended: restart after each install



# 1. Install node.js



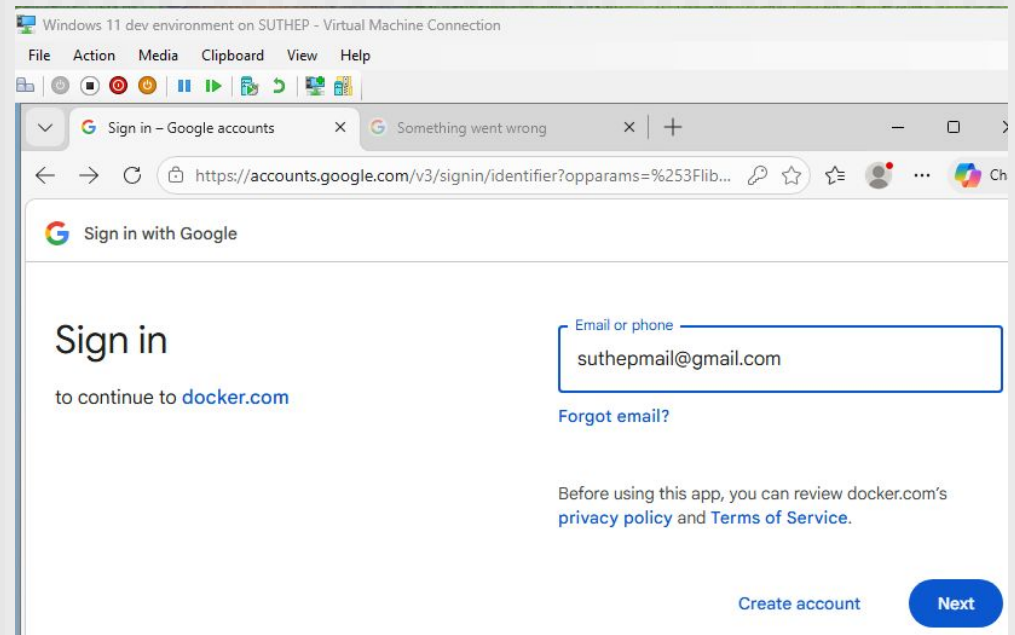


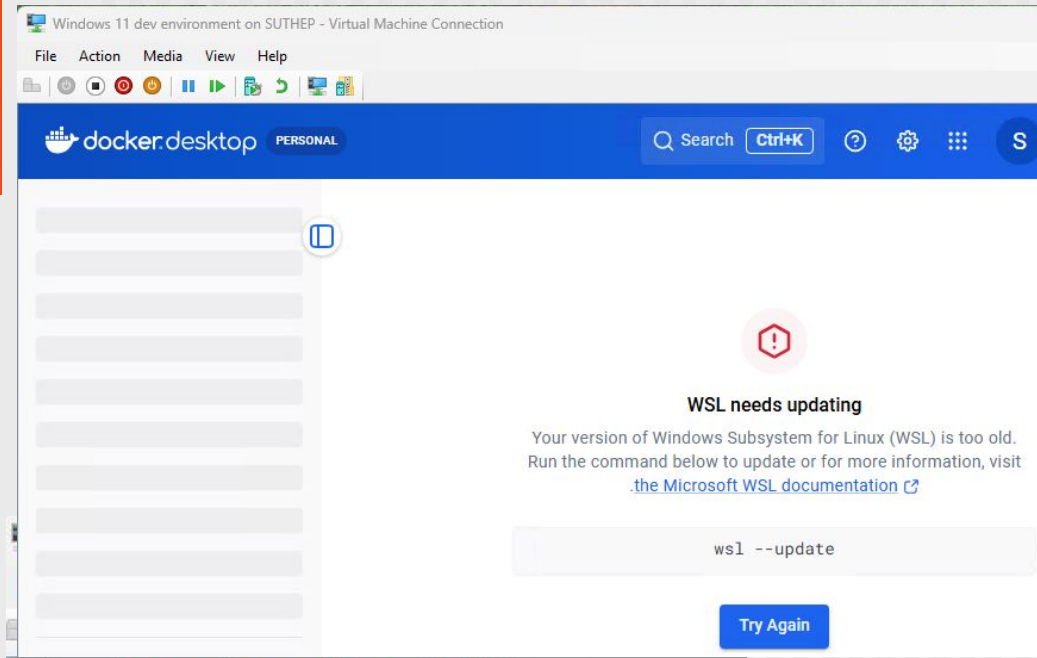
2. Install Docker Desktop. Use default configurations.



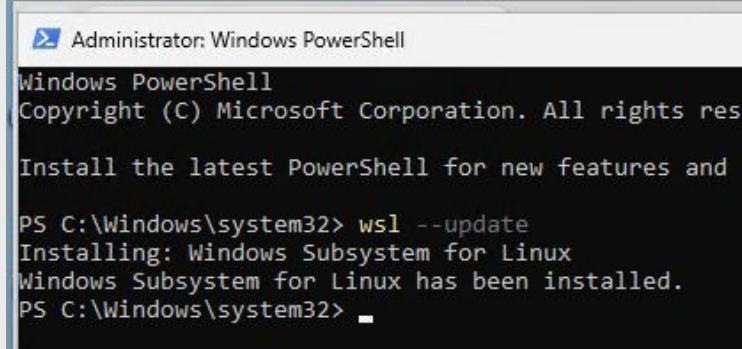
3.1 Run Docker Desktop - click icon

3.2 Sign in with private email





Docker Startup may report error that WSL (Windows Subsystem for Linux) needs to be Updated.



3.3 If get WSL error run as administrator in PowerShell **of your PC (VM if using VM)**

> wsl --update #on VM if use VM

# Install Visual Studio Code (VS Code)

## 4.1 Download VS Code

The screenshot shows a web browser window titled "Download Visual Studio Code - M: X" with the address bar displaying <https://code.visualstudio.com/download#>. The page content includes the Visual Studio Code logo, a navigation bar with a search icon, and a banner for "VS Code February events - Agent Sessions Day on Feb 19th". The main heading is "Download Visual Studio Code", followed by the text "Free and built on open source. Integrated Git, debugging and extensions." Below this, there are three main download sections: Windows (highlighted with a red box), Linux (Debian, Ubuntu), and Mac (macOS 11.0+). Each section has a download button and a list of available installers or packages.

Windows 11 dev environment on SUTHEP - Virtual Machine Connection

File Action Media View Help

vscode install windows - Search x Download Visual Studio Code - M: X

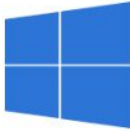
<https://code.visualstudio.com/download#>

Visual Studio Code

[VS Code February events](#) - Agent Sessions Day on Feb 19th

## Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.

↓ Windows  
Windows 10, 11

↓ .deb  
Debian, Ubuntu

↓ .rpm  
Red Hat, Fedora, SUSE

↓ Mac  
macOS 11.0+

User Installer [x64](#) [Arm64](#)

System Installer [x64](#) [Arm64](#)

.zip [x64](#) [Arm64](#)

.deb [x64](#) [Arm32](#) [Arm64](#)

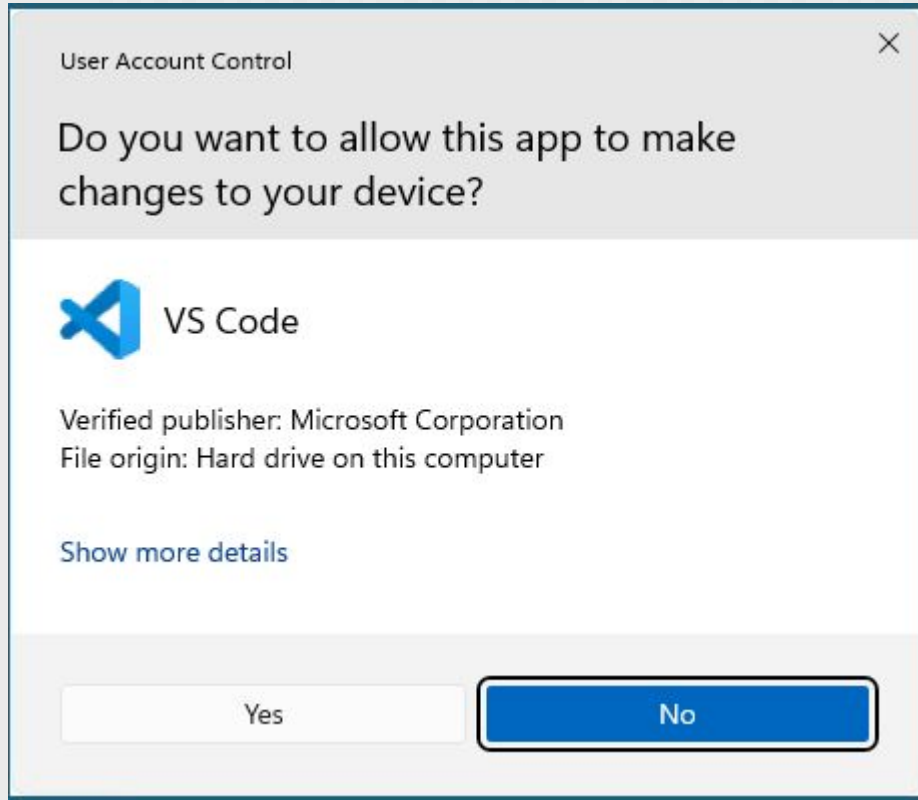
.rpm [x64](#) [Arm32](#) [Arm64](#)

.tar.gz [x64](#) [Arm32](#) [Arm64](#)

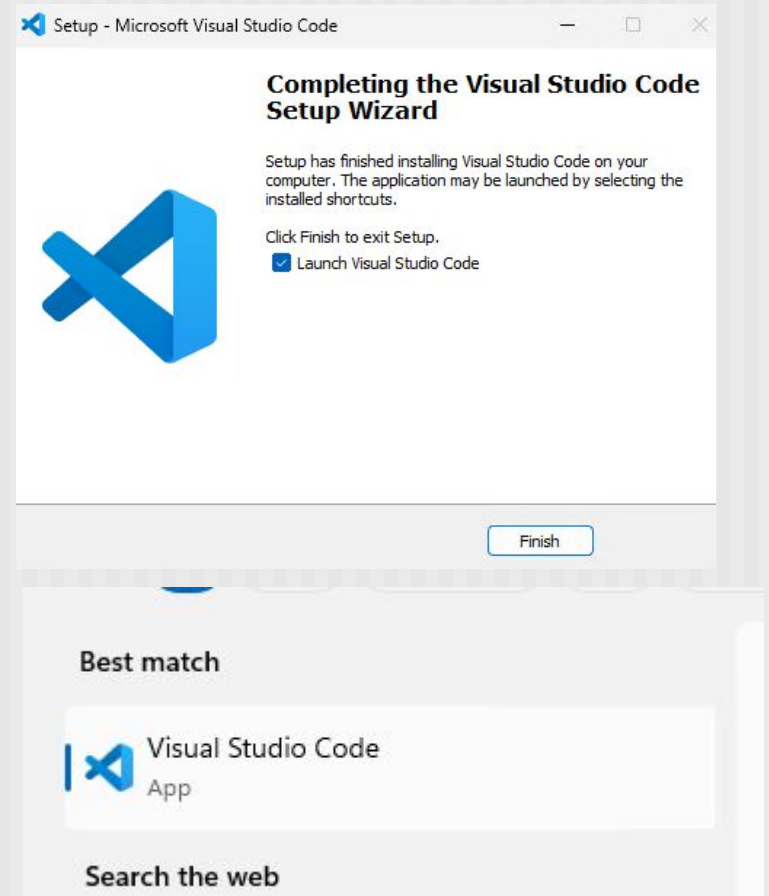
Snap [Snap Store](#)

.dmg [Intel chip](#) [Apple silicon](#) [Universal](#)

CLI [Intel chip](#) [Apple silicon](#)



## 4.2 Install VS Code

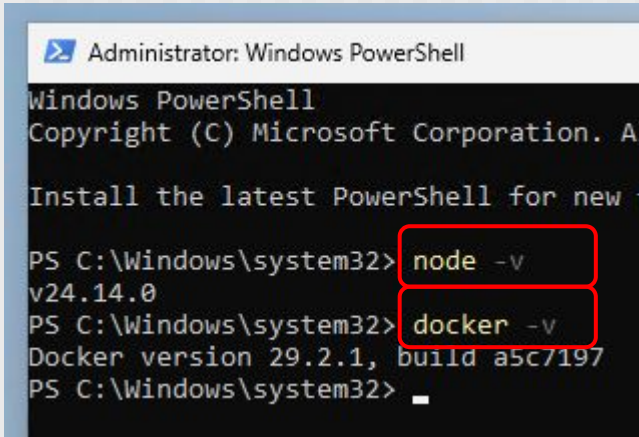


## 4.3 Test run VS Code - click icon

# Quick checks (PowerShell)

---

- > node -v
- > npm -v
- > docker --version
- > docker compose version



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and optimizations, and learn more at https://aka.ms/powershell.

PS C:\Windows\system32> node -v
v24.14.0
PS C:\Windows\system32> docker --version
Docker version 29.2.1, build a5c7197
PS C:\Windows\system32>
```

## **Part 4.**

# Starting up the 3-Tier InvoiceLab App

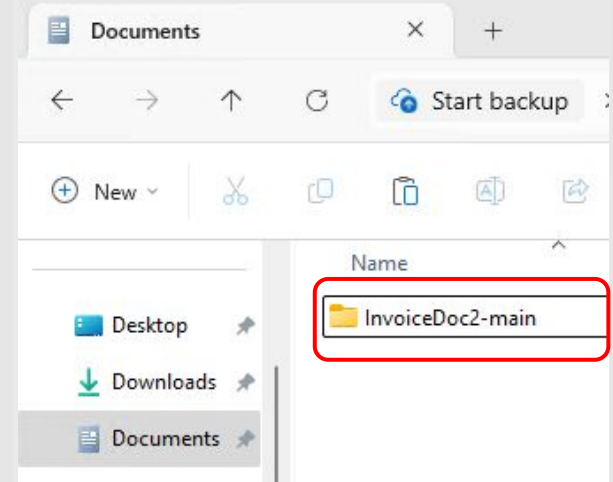
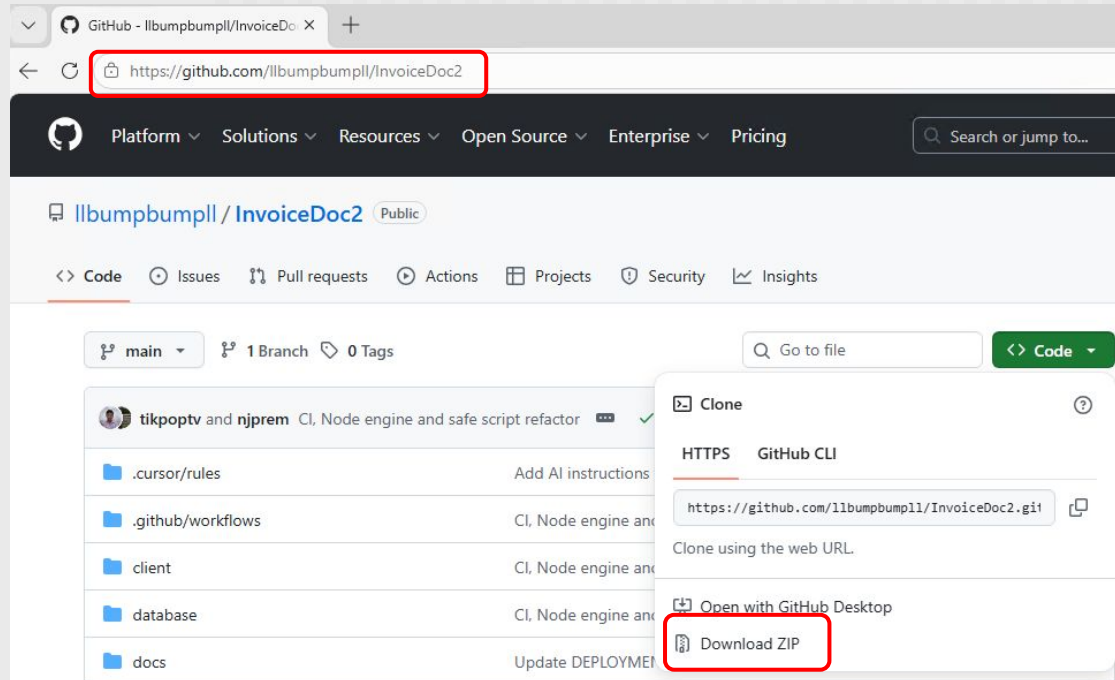
---

# Copy Source Code from the TA's (P' Bump) GitHub Repository

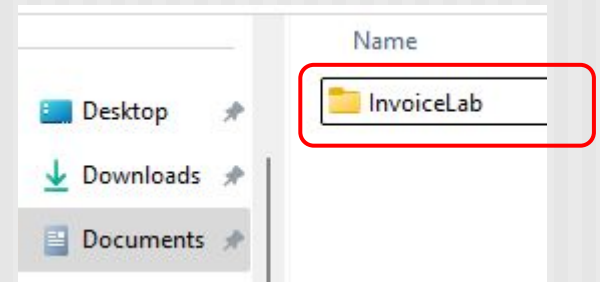
The source code is in GitHub: <https://github.com/llbumpbump11/InvoiceDoc2>

A.1 Download ZIP file from GitHub site

A.2 Unzip folder to Documents. Rename to InvoiceLab.



Rename InvoiceDoc2-main to InvoiceLab





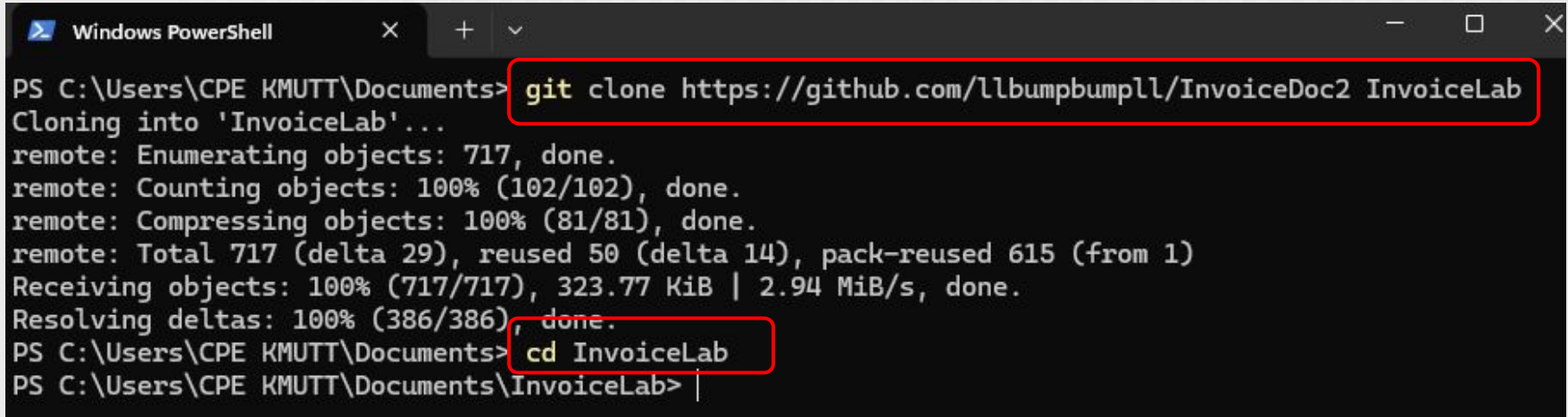
## Better Option: Use git clone to copy repository to your PC

Under a folder such as Documents, clone the repository folder “InvoiceDoc2” into folder “InvoiceLab”:

```
Documents> git clone https://github.com/llbumpbump11/InvoiceDoc2 InvoiceLab
```

```
Documents> cd InvoiceLab
```

```
Documents\InvoiceLab>
```

A screenshot of a Windows PowerShell terminal window. The title bar reads "Windows PowerShell". The terminal shows the following commands and output:  
PS C:\Users\CPE KMUTT\Documents> git clone https://github.com/llbumpbump11/InvoiceDoc2 InvoiceLab  
Cloning into 'InvoiceLab'...  
remote: Enumerating objects: 717, done.  
remote: Counting objects: 100% (102/102), done.  
remote: Compressing objects: 100% (81/81), done.  
remote: Total 717 (delta 29), reused 50 (delta 14), pack-reused 615 (from 1)  
Receiving objects: 100% (717/717), 323.77 KiB | 2.94 MiB/s, done.  
Resolving deltas: 100% (386/386), done.  
PS C:\Users\CPE KMUTT\Documents> cd InvoiceLab  
PS C:\Users\CPE KMUTT\Documents\InvoiceLab> |  
In the original image, the command `git clone https://github.com/llbumpbump11/InvoiceDoc2 InvoiceLab` and the command `cd InvoiceLab` are highlighted with red rectangular boxes.

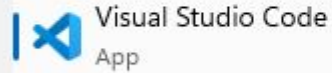
# What's in the InvoiceLab Repository

---

1. **database/** = Dockerized PostgreSQL + Adminer (port 15432)
2. **server/** = Express API backend (default port 4000)
3. **client/** = React + Vite UI frontend (default port 5173)
4. **Default DB URL:** postgresql://root:root@localhost:15432/invoices\_db
5. **Default API base:** http://localhost:4000
6. **Default API base:** http://localhost:5173

## Run VS Code and Open Folder InvoiceLab

Best match

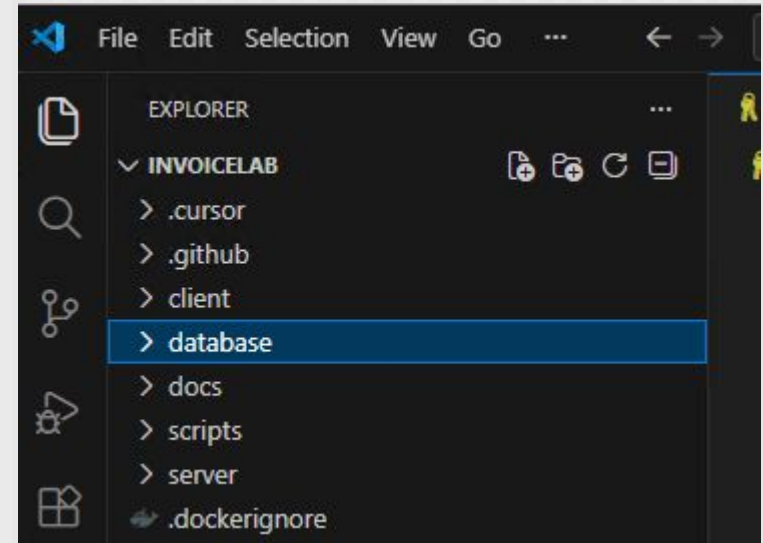
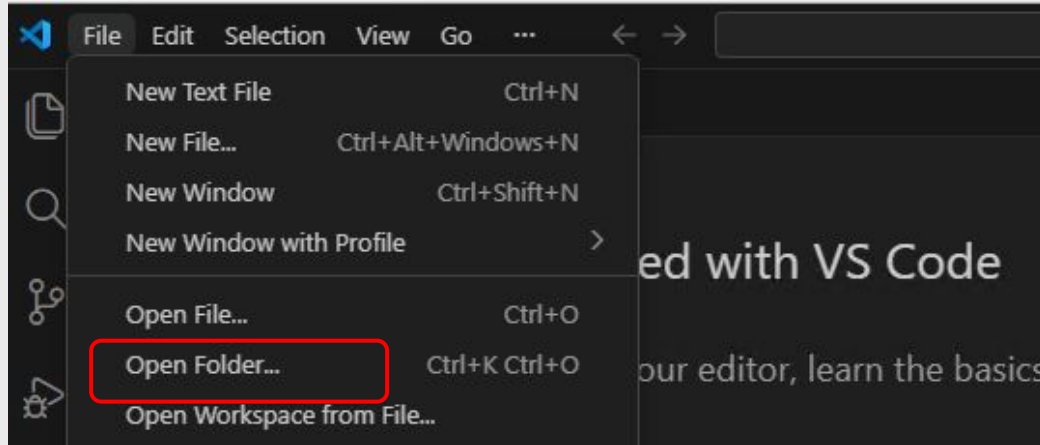
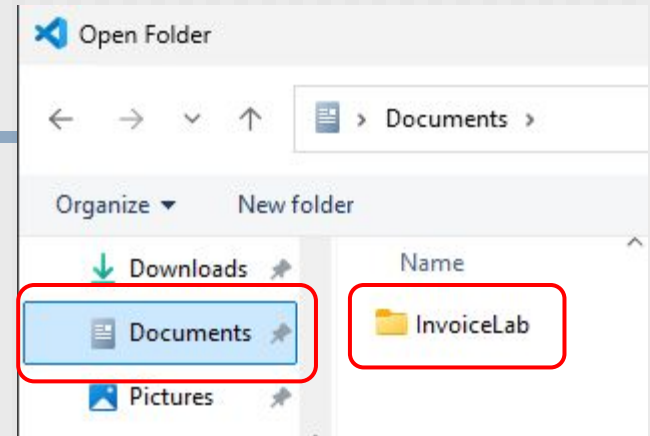


Search the web

B.1 Run VS Code (icon)

B.2 Open Folder ...

B.3 Select InvoiceLab folder



# Run the 3 tiers locally (overview)



1. **Step 1:** Start Docker Desktop (windows icon)
2. **Step 2:** Start DB from repo root (runs Postgres server + seeds data) in Terminal 1  
**InvoiceLab>** `npm run docker:db:start`
3. **Step 3:** Run API (server) in Terminal 2
4. **Step 4:** Run UI (client) in Terminal 3
5. **Step 5:** Open UI and test

## Optional Step 2.1:

- To get a Postgres CLI (psql terminal) inside the container of Terminal 1:

```
InvoiceLab> docker compose -f database/compose.yaml exec -it pgdatabase psql -U  
root -d invoices_db
```

```
invoices_db=> \dt -- to list tables
```

```
invoices_db=> \d invoice -- describe invoice table (schema)
```

```
invoices_db=> SELECT * FROM invoice; -- shows data rows in invoice table
```

# The optional PostgreSQL psql terminal

```
InvoiceLab> docker compose -f database/compose.yaml exec -it pgdatabase psql -U root -d invoices_db
```

- “Go inside the running Postgres container and open an interactive PostgreSQL terminal (psql) connected to the `invoices_db` database as user `root`.”
- **psql terminal:** interactive terminal tool (command-line client) that lets you talk to Postgres.

```
> psql -U root -d invoices_db #windows command line to open a psql terminal
```

Inside that session you can:

- Run normal SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`, etc.)
- Run **psql meta-commands** that start with `\` such as (these are psql and not SQL commands):
  - `invoices_db=> \dt` -- list tables
  - `invoices_db=> \d invoice` -- describe table invoice
  - `invoices_db=> \prompt` -- ask user for input

# Adminer: Another optional DB management tool

---

**Optional** : Start Adminer by entering the following in another command line (Terminal 4):

```
InvoiceLab> docker compose -f database/compose.yaml up -d adminer
```

**Adminer Functions:** Connect to a database (such as PostgreSQL), view tables, run SQL queries, and inspect or modify data **without using the command line**.

Adminer can provide a convenient web UI to explore the **invoices\_db** database running in Docker.

- **Adminer** = graphical, point-and-click, beginner-friendly
- **psql** = command-line, faster for power users, better for scripting and writing raw SQL

**Adminer URL:** <http://localhost:8080>

## Step 3. Run API in Terminal 2.

---

1. From project root go to server/ folder

```
InvoiceLab> cd server
```

2. Install dependencies once

```
InvoiceLab\server> npm install
```

3. Create .env from .env.example

```
InvoiceLab\server> Copy-Item .env.example .env    # Windows  
OR
```

```
InvoiceLab/server$ # cp .env.example .env    # macOS/Linux
```

4. Start dev server

```
InvoiceLab\server> npm run dev
```

## Step 4. Run UI in Terminal 3.

---

1. From project root go to client/ folder

```
InvoiceLab> cd client
```

2. Install dependencies once

```
InvoiceLab\client> npm install
```

3. Create .env from .env.example

```
InvoiceLab\client> Copy-Item .env.example .env # Windows
```

**OR**

```
InvoiceLab\client> # cp .env.example .env # macOS/Linux
```

4. Start React + Vite dev server

```
InvoiceLab\client> npm run dev
```



# Open and test your running app

---

1. **UI:** `http://localhost:5173`
2. **API health:** `http://localhost:4000/health`

UI → API → DB

**Ports:** UI 5173 | API 4000 | DB 15432 | Adminer 8080

## Next time you start your system



1. Start Docker Desktop (click icon)
2. Terminal 1 (DB Server):  
**InvoiceLab>** `npm run docker:db:start #at repo root`
3. Terminal 2 (API server): `npm run dev`  
**InvoiceLab>** `cd server #at repo root`  
**InvoiceLab\server>** `npm run dev`
4. Terminal 3 (client UI): `npm run dev`  
**InvoiceLab>** `cd client #at repo root`  
**InvoiceLab\client>** `npm run dev`

Run npm install again only if first setup or dependencies change

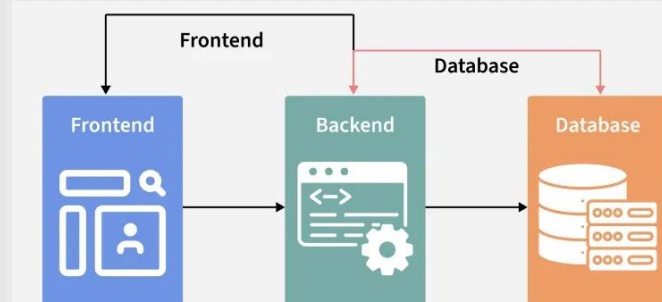
# Quick API test via a PowerShell terminal

---

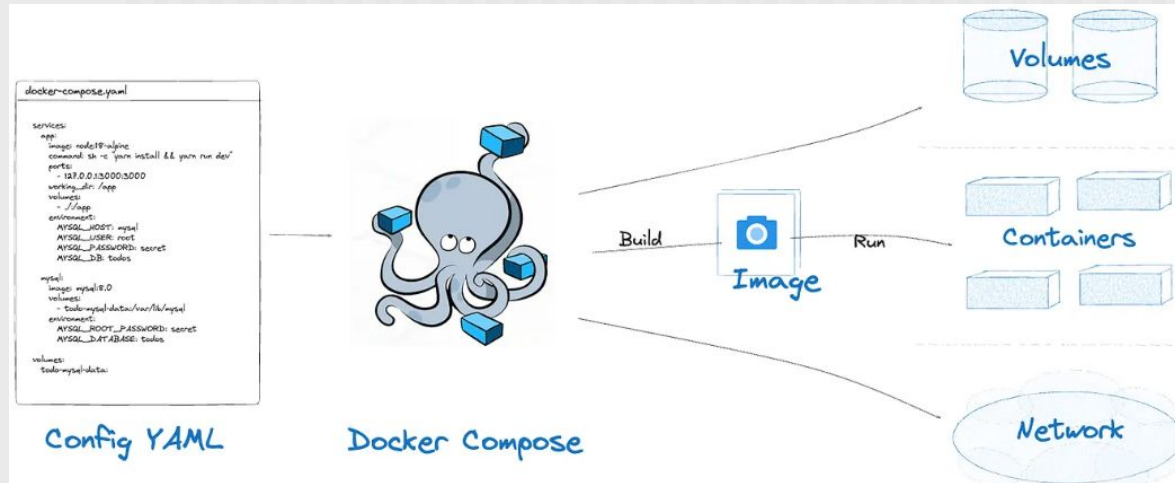
```
> Invoke-RestMethod "http://localhost:4000/api/invoices?page=1&limit=5"  
| ConvertTo-Json -Depth 6
```

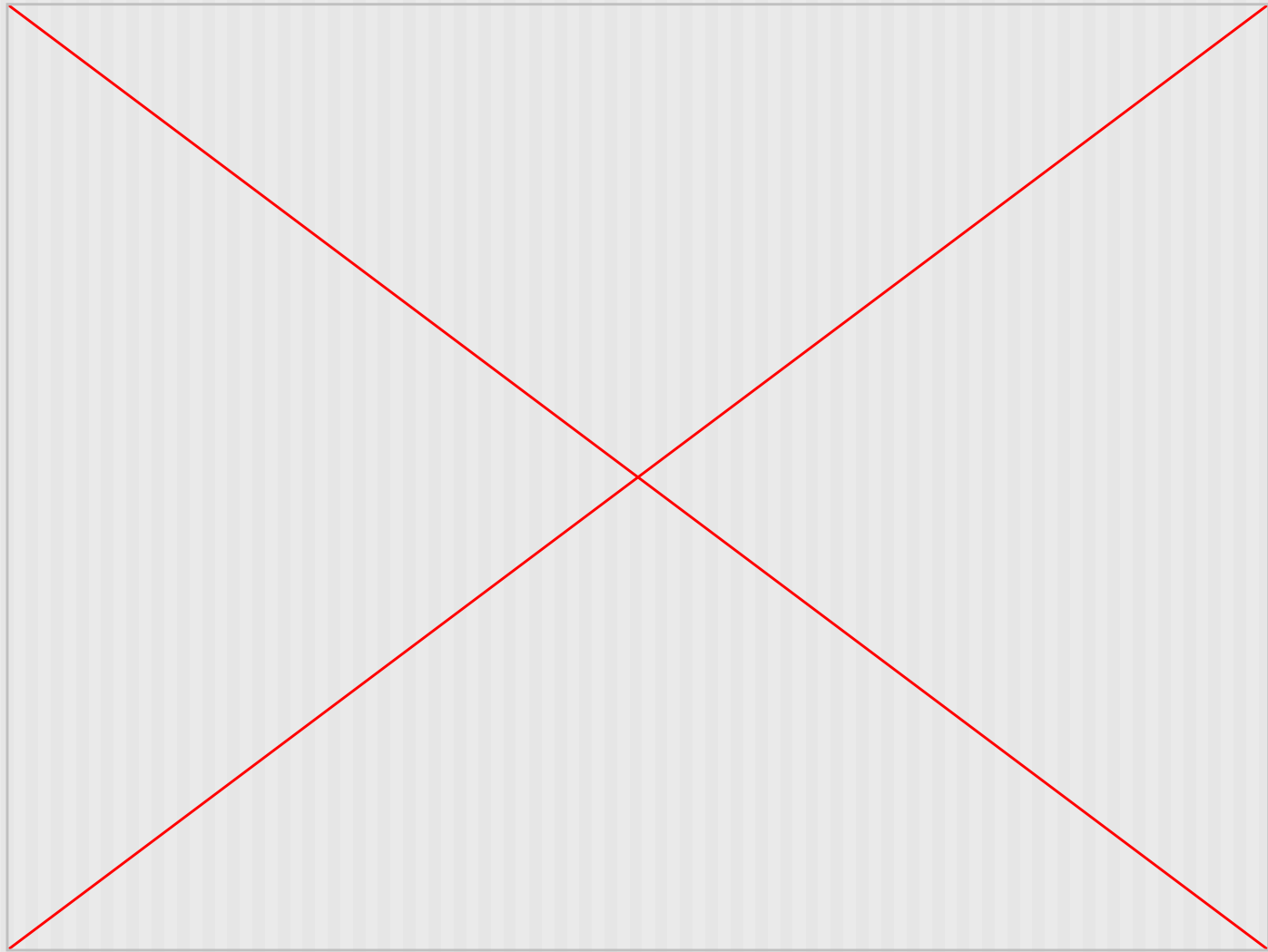
# FYI: Full stack in Docker Option (not supported)

1. DB + server + client containers
2. **Start:** `> npm run docker:start`
3. **UI:** `http://localhost:3000`
4. **API:** `http://localhost:4000`



Docker Compose environment with 3 docker containers. Compose does the orchestration.





## Part 5.

# Test run of InvoiceLab App

---

